



Project Brief 2026

Introduction

This brief provides details for the Software Development I Project. It consists of several parts. The first part describes in general what you are required to do. The second part presents a detailed specification of how data will be presented to your program and what you will be required to generate as an output. The third and final part discusses the deliverables that must be submitted before the project hand-in deadline.

Important information:

- The deadline for submission of the final project deliverables is **08h00 Tuesday 28 May 2026**. Late submissions will be penalised as described in the School's policy on late submissions.
- The project contributes 15% to the final mark of the course.
- Full details about final deliverables and assessment criteria are included in this project brief.
- Although the project is a group submission, please be advised that cheating and plagiarism will not be tolerated - this applies to both source code and written reports. Any suspected cases of cheating or plagiarism may be referred to the legal office of the University for further investigation.
- Although you may make extensive use of the Internet in the background and research component of the project, be sure to only make use of respected online references in the final report.
- At the end of the project, you will be required to provide a comprehensive timesheet which includes a complete breakdown of both the original estimated time as well as the actual time spent. It is therefore critical to establish a disciplined approach right from the start of the project (hint: make use of an engineering notebook as well as a tool such as a spreadsheet for logging time. There are also free apps available for time logging, such as Toggl: <https://toggl.com/>).

Part 1: Background

Morabaraba is a two-player strategy board game that originated in Southern Africa. The game is played on a board with 24 holes and 12 pieces for each player. The pieces are usually black and white stones or beads which also referred to as cows. The objective of the game is to form "mills" of three pieces in a row, either along the lines of the board or around the circles. A player who forms a mill can capture one of the opponent's pieces, removing it from the board. The game has three phases: placement, movement and flying. In the **placement phase**, players take turns placing their pieces on any empty hole on the board until all 24 pieces are placed. In the **movement phase**, players take turns sliding one of their pieces along a line to an adjacent empty hole. In the **flying phase**, which begins when a player has only three pieces left, players can move their pieces to any empty hole on the board without following the lines. The game ends when one player has

only two pieces left or cannot make any legal move. If either player has only three pieces/cows and neither player eliminates a cow within ten moves, the game is a draw. You are required to develop a computer program in C++ that allows **two different algorithms** to compete against each other in a **game of Morabaraba**.

The standard Morabaraba board consists of 3 squares each having 8 intersections as shown in Fig. 1a.

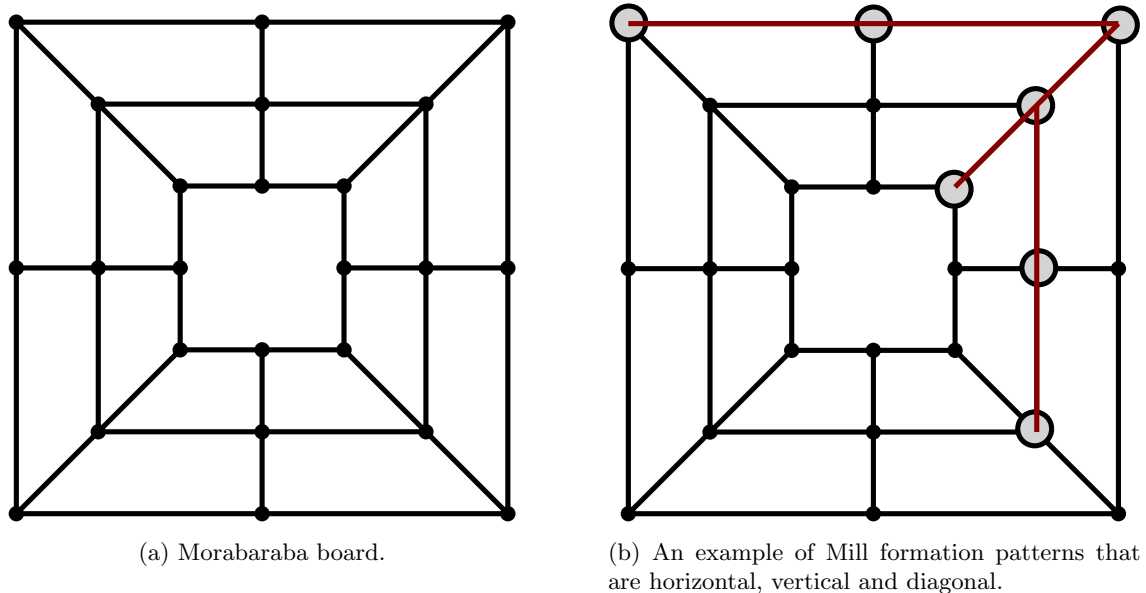


Figure 1: A basic Morabaraba board.

A summary of the rules based on the three phases of the game is provided below

Placement phase

- At the start of the game, there are no pieces on the board. Each player has 12 pieces, known as “cows”; one player has white cows and the other has black cows.
- The game is played in turns, with each player placing a cow on an unoccupied intersection of the board during their turn.
- The objective is to form a “mill”, which is a row of three cows on any line drawn on the board.
- When a player creates a mill, they are allowed to eliminate one of the opponent’s cows. The cow is taken off the board and cannot be returned. A cow in a mill cannot be eliminated unless all of the opponent’s cows are in mills; in such a situation, any cow can be removed from the board..
- It is important to note that, regardless of whether a move results in the formation of multiple mills, only one cow can be eliminated per turn.

Moving phase

- Once all cows have been placed on the board, each turn involves moving a cow to an adjacent and unoccupied intersection.
- A mill can be broken and reconstructed multiple times by moving cows back and forth. However, if a player breaks one of their mills, they cannot reform the same mill in the following turn using the same cows in the same positions.

Flying phase

- If a player has only three cows remaining on the board, they are allowed to “fly” to any unoccupied intersection, not just adjacent ones.
- If one player has three cows left and the other player has more than three cows, only the player with three cows can fly their cows.

Finishing the game

- The game is won if the opponent cannot make any move.
- The game is won if the opponent has only two cows left.
- If either player has only three cows and neither player eliminates a cow within ten moves, the game is drawn.

The easiest way to become accustomed with the rules of the game is to watch the video in the link below. Visit https://www.youtube.com/watch?v=kj1Fp13Lifc&ab_channel=NewVentureGames. Please feel free to look for the online version/apps of the game as a way to properly familiarise yourself with the rules. You can also physically set up the game and play with your classmates.

Project Requirements and Constraints

In this project, you are required to

1. Implement Morabaraba for a board that consists of 3 squares each having 8 intersections.
2. Implement two different algorithms that will play Morabaraba against each other. The selected algorithms will exclusively be referred to as Alg_1 (player 1) and Alg_2 (player 2), **no other names will be accepted**. For example:
 - (a) Alg_1 could be a random move selection strategy.
 - (b) Alg_2 could be a heuristic-based AI that chooses the best move to reduce tile values efficiently. tile values efficiently.

Please note, since this is a group project, each group member must implement one algorithm. The final code should successfully run with both algorithms. **Failure to meet this condition will result in a penalty to both group members.**

3. The two algorithms will play against each other on the same board at the same time. The first algorithm to complete the game will be considered the winner.
4. The game should be played in 4 different variations based on the number of pieces (cows) each player starts with. Player should start the game with either 4 pieces each, 6 pieces each, 9 pieces each or 12 pieces each.
5. To decrease the likelihood of a draw, the flying phase of the game should be ignored in your implementation of the code.

This project will expose you to several key aspects of software development and algorithm design, including:

- The (software) engineering lifecycle: problem identification, requirements analysis, design, implementation, testing and verification, documentation and maintenance.
- Algorithm design and optimization: Choosing an efficient approach to minimize moves.
- State management of the program: Handling board updates and detecting possible moves.

- Heuristic strategies: Implementing an automated solver to optimize gameplay
- Collaboration, Project planning and time management: Teamwork within your groups, Managing resources and testing different solving techniques.

Take time to research and find out more about the above.

1.1 Project and Time Management

Project and time management are essential skills that must be consistently developed and refined, especially in software development. As the saying goes “time costs money” - failure to accurately predict in advance the time spent on a project can be very costly. There are many examples of large software project disasters where deadlines were not met.

It is therefore essential that you develop good habits as early as possible, for example, always make use of an engineer’s notebook and maintain a comprehensive spreadsheet that contains a breakdown of the time you spend on various activities within a project. Get into the habit of estimating the amount of time you think a task will take and then retrospectively comparing the prediction against the actual time spent. Critically analyse these times, even if the actual time was exactly what you predicted it to be. With practice and experience you will develop an important skill.

As a starting point, you may wish to use the following broad software development activities and the proposed time breakdown:

Activity	Time Breakdown
Background (problem understanding, requirements)	15%
Analysis & Design	20%
Implementation (coding)	20%
Testing	20%
Documentation	25%

Another important piece of information to consider is that each second year student taking Software Development I is expected to spend 12.5 hours every week on the course (including lectures, laboratories and tutorials).

Given the above information, as well as the deadline for the submission of project deliverables, you should be able to formulate a suitable time plan for your project. You may use the breakdown introduced in the previous section, or propose your own. For each major activity, list the time that will be spent on the activity as a percentage of the total time, as well as the actual number of hours. You must also provide the total number of hours you plan to spend on the project.

Part 2: Data and Specifications

You **MUST** ensure that you follow **ALL** of the instructions, and meet all the specifications and requirements, provided in this document. Failure to do so will result in certain assessment outcomes being rated *Poor* or *Unacceptable*.

You are required to develop a computer program in C++ that allows two different algorithms to compete against each other in a game of Morabaraba .

Your code must meet the following specifications:

- Read in a list of integers specifying the number of pieces used by each player.
- Algorithm 1 will take alternating turns with algorithm 2 at placing their pieces/cows on the game board.
- A piece/cow cannot overwrite an already filled position in the grid.
- The list of moves made by each algorithm must be stored to an output file.

2.1 Board format

For you to record the moves made by your algorithms in a meaningful way that can be understood by others (and more importantly the person marking your project) there must be a standard labelling system used by everyone.

As mentioned in the background section, a standard Morabaraba board consists of 3 squares (S_0, S_1, S_2) each having 8 intersections ($i_0, i_1, i_2, i_3, i_4, i_5, i_6, i_7$) which will be labelled as shown in Fig. 2.

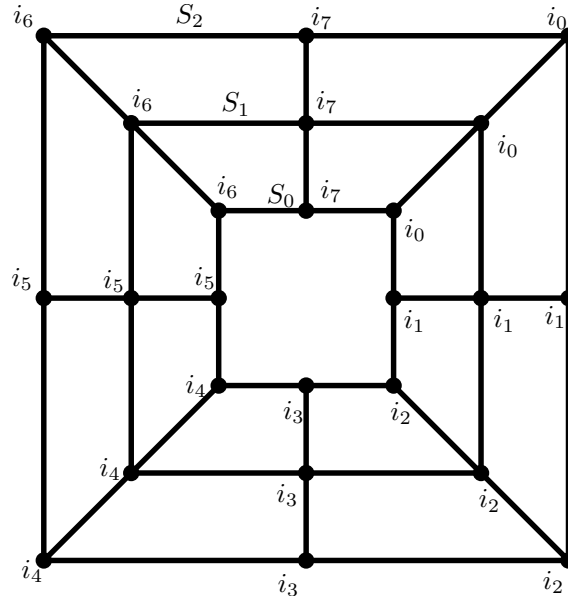


Figure 2: A labeled Morabaraba board.

The player can start from any point on the board.

2.2 Input format

The file `input.txt` will include a series of numbers separated by line breaks, which will indicate the number of pieces that each player will start with. These numbers will be used to test the two algorithms playing against each other. The input file will have the following format

```
4
6
9
6
12
```

Please note that the input file can contain the number of pieces in any order. You can assume the file will contain any of the specified numbers of pieces, either 4, 6, 9, and 12. Tip: To avoid complications, it is recommended to test the game implementation with an input of 4 pieces first as it is easier to debug compared to working with 12 pieces.

2.3 Output format

Your program must produce one output file. The file must be named `morabarabaResults.txt`. This file must contain the following:

- The first line of the output should indicate the number of pieces or cows that both players have at the start of the game.

- Each line after this should record a players turn - list “Alg_1” or “Alg_2” depending on whose turn it is.
- In the placement phase, player moves are denoted as “00-Sx,iy”. Here, “00” represents that the piece/cow has been moved from outside the board, and “Sx,iy” denotes the current position of the piece/cow. The value of Sx (for $0 \leq x \leq 2$) is the square number, and iy (for $0 \leq y \leq 7$) is the intersection number. For instance, “Alg_1 00-S1,i1” means that player 1 moved the piece/cow from outside the board to intersection 1 on square 1 (see Figure 2 for the position reference).
- In the movement phase, player moves are denoted as “Sx_p,iy_p-Sx_c,iy_c”. In this format “Sx_p,iy_p” denotes the previous board position and “Sx_c,iy_c” denotes the current board position of the piece/cow. For instance, “Alg_1 S1,i7-S1,i0” means that player 1 moved the piece/cow from intersection 7 on square 1 to intersection 0 on square 1 .
- **The project implementation of this game will not include the flying phase, in order to decrease the chances that the game ends in a draw.**
- If a player creates a mill during their move, your code should display a message alongside the move that updates the current state of the game. The message should include the positions of the three pieces/cows that formed the mill, the position of the cow that the opponent will lose, and how many cows the opposing player has left. For example, in line 9 of Listing 1, the following message is displayed::

```
Alg_2 00-S2,i4 (mill = S2i4, S2i5, S2i6) Alg_1 losses cow (S1,i0) | 3 left
```

This indicates that player 2 moved the piece/cow from outside the board to intersection 4 on square 2, which resulted in a mill being formed at positions (S2,i4), (S2,i5) and (S2,i6). Player 1 lost a cow at position (S1,i0) and now has 3 cows left

- The final piece of information to include is who the winner is or if it is a draw.
 - If there is a winner write either “Alg_1 wins” if player 1 wins or “Alg_2 wins” if player 2 wins.
 - If there is a draw, write “its a draw”.
- Any subsequent games listed in this file should be separated from previous games by an empty line as shown in line 15 of Listing 1.

Only the number of pieces/cows listed in the current input file must be saved to the output file. The history for each game based on the different number of pieces listed in the input file must be appended to this file with one empty line in between different games.

Listing 1: An example of the format for morabarabaResults.txt.

```

1  4
2  Alg_1 00-S1,i1
3  Alg_2 00-S2,i6
4  Alg_1 00-S1,i0
5  Alg_2 00-S1,i6
6  Alg_1 00-S1,i2 (mill = S1i0, S1i1, S1i2) Alg_2 losses cow (S1,i6) | 3 left
7  Alg_2 00-S2,i5
8  Alg_1 00-S1,i6
9  Alg_2 00-S2,i4 (mill = S2i4, S2i5, S2i6) Alg_1 losses cow (S1,i0) | 3 left
10 Alg_1 S1,i6-S1,i7
11 Alg_2 S2,i6-S1,i6
12 Alg_1 S1,i7-S1,i0 (mill = S1i0, S1i6, Si7) Alg_2 losses cow (S2,i4) | 2 left
13
14 Alg_1 wins
15
16 4
17 //output of remaining games would appear here

```

Note you must use the exact format above - it is case sensitive, spacing sensitive and the names Alg_1 and Alg_2 must be used. In your report you can elaborate on what each players' algorithm is. There should be no console output produced in your final submission.

Part 3: Deliverables and Submission

Project Submissions Overview

The project will have **three** main submissions. The three submissions are designed to ensure proper planning and implementation of the program throughout the software development lifecycle. Each submission will represent a step forward in the ongoing development of the program and provide iterative progress towards the completion of the project. The first two hand-ins are work-in-progress submissions and a reduced set of project deliverables is required. The final hand-in will require the submission of all of the project's deliverables. Below, please find the full description of the three submissions

Submission 01: Summary of the project plan

This submission should include a one-page description of your proposed solution and will be referred to as the project plan. It should detail the algorithms you intend to use, explaining why they were selected for the solution, along with a description of the functions that will be implemented in the program. Additionally, a flowchart should be included on a separate page together with the project plan.

Please take note of the information below for Submission 01.

- **Submission 01 Deadline: 30 March 2026 @ 08h00.**
- This is a group submission, so only one member of the group should make the submission. However, it is the responsibility of each group member to ensure that the submission is made on time.
- Both the project plan and flowchart submissions must be in PDF format and contained within a single compressed .zip file. Any other file format will not be accepted. The submitted zip file should be named using the format

FirstnameLastname_Studentnumber.zip.

- No submission made outside of the Ulwazi portal will be accepted.
- No late submission will be accepted. Please note that the school policy on late submissions does not apply to this submission.
- Failure to make this submission will result in a 5% deduction from your final project mark.

Submission 02: Basic Board Functionality

This submission requires a code submission with the basic board implementation of the Morabaraba game. The expected functionality of the code is listed below.

- a board for the required game size.
- Any kind of movement of piece/cow on the board. The piece/cow movement should be presented on the terminal. Feel free to initiate the piece/cow movement either by using an input file or by requesting user input.
- Please submit a README file together with the code to provide instructions on how to run the basic tile movement.

- No class implementation is required for this submission, but at least one function implementation is mandatory.

Please take note of the information below for Submission 02.

- **Submission 02 Deadline: 13 April 2026 @ 08h00.**
- This is a group submission, so only one member of the group should make the submission. However, it is the responsibility of each group member to ensure that the submission is made on time.
- Electronic submission of project source code (.cpp) contained within a single compressed .zip file. The submitted zipped source code should be named using the format

FirstnameLastname_Studentnumber.zip.

- No submission made outside of the Ulwazi portal will be accepted.
- No late submission will be accepted. Please note that the school policy on late submissions does not apply to this submission.
- Failure to make this submission will result in a 5% deduction from your final project mark.

Submission 03: Full working code and Report

This submission should include a full working code based on project requirements and a report. Below please find the specifics of the submission

- **Submission 03 Deadline: 28 April 2026 @ 08h00.**
- Electronic submission of project source code (.cpp and any .h files required for the code to compile)
- The source code should include at the minimum one class and by extension at least one function.
- Electronic submission of a PDF version of project report.
- If you have made use of any external software libraries or have used multiple source code files, be sure to include these in your online submission. All source code files must be contained within a single compressed .zip file, named using the format

FirstnameLastname_Studentnumber.zip.

- The report and the source code will have separate submission portals. Links will be provided on Ulwazi for submission of your source code and PDF report online.
- This is a group submission, so only one member of the group should make the submission. However, it is the responsibility of each group member to ensure that the submission is made on time.
- No submission made outside of the Ulwazi portal will be accepted.
- Failure to submit either of the above Submission 03 deliverables before the deadline will result in the overall project mark being subject to a penalty as described in the school policy on late submissions (refer to the “Red Book”). Any exceptions must be discussed by the student with the Head of the School of Electrical & Information Engineering.

3.1 Code Requirements

The code requirements of the project submission are as follow:

- No global variables may be used - this will result in an automatic failure of the project
- Use of at least one class and by extension at least one user defined function.
- Reading in and writing to files.
- There should be no console output for the final submission.

3.2 C++ source code

Your source code will be assessed by looking for evidence supporting the successful engineering execution of the project. This means that coding style, use of comments, clarity of code, ability to compile, run and generate correct output and adherence to specifications and requirements are important factors.

The first line of your C++ source code must be a single line comment with the format (replace with your own student number):

```
// <StudentNumber>
```

As with any formal assessment in the University you are not required to include your name with your submission, however, it is highly recommended that you do so. You should also include any other useful information as comments at the top of your source code submission. Failure to include your student number will result in the “Background & Problem Understanding” assessment outcome being rated as Poor or *Unacceptable*.

Note that the input data file your program makes use of must be named `input.txt`.

Your code will be marked on a machine running Windows with g++ version 14.2.0. Ensure that your code includes any necessary headers correctly. You can check this by compiling your code via the command line (on Windows, Ubuntu or Mac OS). Open the command prompt window from within your source code directory (or navigate there) and type `g++` followed by a list of all source code files you have created. For example, to compile and run your code in the Windows Command prompt, you use the command:

```
g++ main.cpp class.h -o run & run
```

To compile and run your code on Mac or Linux using the Terminal, you use the command:

```
g++ main.cpp class.h -o run && ./run
```

When writing source code, make sure to practise good programming techniques as discussed during the course. In particular, pay attention to variable naming, comment your code, be consistent with indentation, avoid global variables at all costs, avoid placing too much code in `main()`, include thorough error checking and validation, and make use of well thought out modules and/or classes to solve the problem effectively.

Remember that careful planning and time management is critical to a successful project - at all stages keep ongoing notes about your activities and time spent for your time management record.

3.3 Documentation requirements

An important deliverable of the course project is a written report which provides tangible evidence that you have engineered a successful solution to the problem you were given. The body of this report along with your source code needs to provide evidence towards each of the following outcomes:

- Background and problem understanding
- Quality of engineering output
- Critical analysis and evaluation

Note, however, that this does not mean that your report should have the above items as section headings! The body of your report, excluding title page and appendices, **must be no longer than four pages**. Note that appendices must be self-standing documents.

The report must be submitted as a single electronic Portable Document Format (PDF) document. Microsoft Word has a built in export to PDF function that can be used when saving your document.

The completed documentation must be concise and of a high standard, in accordance with the accepted guidelines for report writing and presentation delivering that apply within the School of Electrical and Information Engineering (see the blue book).

Your report must meet the following specific requirements:

- It must be submitted electronically as a single PDF document via the Wits Sakai system. You are not required to submit a printed copy. Submit your report to the project report submission link.
- The report must consist of the following sections:
 - A Title: for example “A simple strategy for Morabaraba”, your name and student number, the branch of Engineering you are registered and the date on the top of the first page. A title page should be included with your report.
 - A short abstract (50 words approximately)
 - Section 1: Introduction (see blue book)
 - Section 2: Design

This section details the planning of your solution. Ideally this section would have been completed before a line of code was written. Detail any assumptions made prior to implementation and motivate them. Provide an overview of the major modules involved in your solution and provide a detailed flow chart which illustrates the operation of your program (two pages or less). The flow chart must illustrate file handling and all algorithms. The focus of this section should be descriptive rather than technical. Note that the objective of this section is for someone to be able to follow what the program does and why it does it (the how is taken care of by the source code).
 - Section 3: Results

Each student is required to test their program using data which can be downloaded from the website. You must run your program multiple times using different data sets. Using the output files created, as well as other tools (such as a spreadsheet), you should generate various tables and graphs that can be presented in your report. These results can be used to motivate which algorithm performs better (one page or less).
 - Section 4: Discussion

Critically analyse and discuss your approach, solution and findings. For example, what conclusions can be made about the two implemented algorithms? Were your initial assumptions valid? What improvements can be made to your approach or solution?
 - Section 5: Conclusions

As per the “Blue Book”.
 - Section 6: References
 - Appendix A: Project Time Management

Predicted versus actual with a brief critical discussion (maximum two pages).
- The body of your report (from the Introduction to the end of the References) **may not exceed four pages** – additional pages will not be considered.
- You may include your flowchart as an appendix.

Final notes

- There is no provision for user input for the final submission of your program, it must act on and produce files as specified in Part 2 and Part 3 without any human interaction!
- If you include multiple files in your submission please ensure you include a README file (find out what this is).
- Students aiming for good results will want to satisfy all the requirements mentioned in the previous sections, as well as tackling additional issues which should be highlighted in the final report.
- Do not include any input or output files in your submission! Also, submission of additional project files (executables, IDE project files, etc) is unacceptable, you must only submit the source code (`.cpp` files and `.h` files).

NB: Your output files must be named as specified in Part 3. Pay attention to issues related to case-sensitivity. Your report must be submitted as a single PDF document.

Your source code must be submitted as separate files.

Important: Check and double-check that your project submission is successful on the online system.